SAMSUNG

Samsung Innovation Campus

Generative AI Module

Together for Tomorrow!
Enabling People
Education for Future Generations

Notas de Clase

Generative AI Module

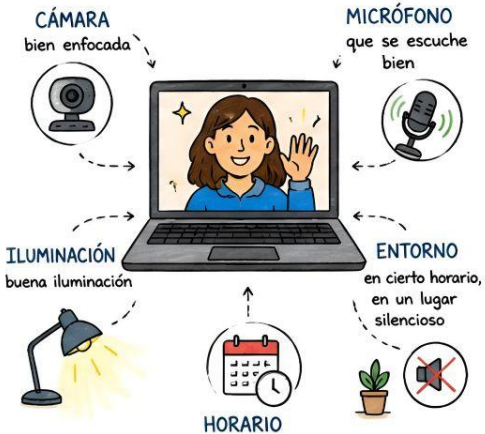
OBJETIVOS DEL CURSO

1 Se lleven como razonar un problema y saber cuando usar AI y cuando no.



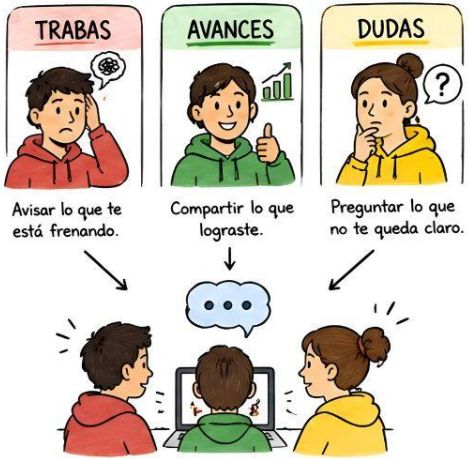
Primero pensar, después decidir.

2 Organizar su vida para tener meets, eso implica tener tu carta de presentacion en buenas condiciones:



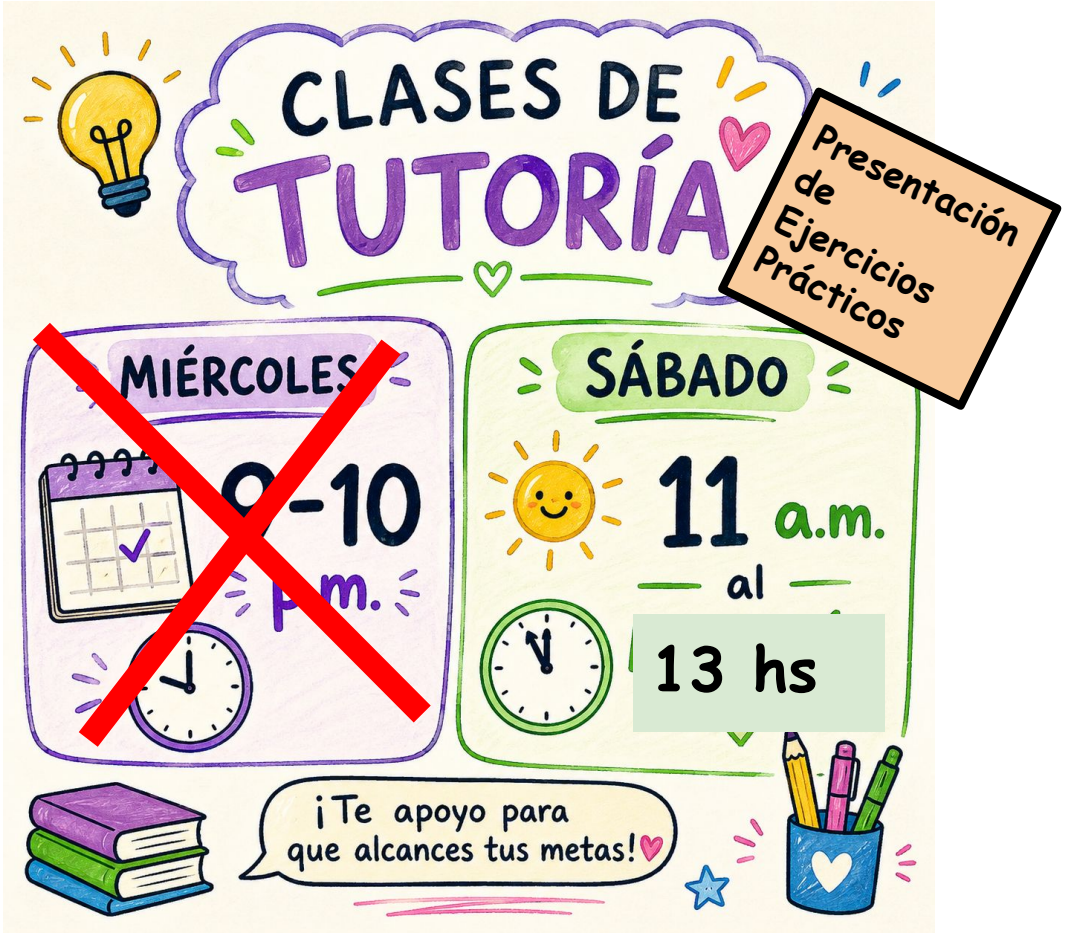
Tu meet es tu carta de presentación. ¡Cuidala!

3 Poder comunicar trabas, avances y dudas a tiempo, es preferible equivocarse aca que con una deadline.



Comunicar a tiempo = mejores soluciones, menos estrés.

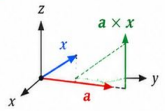
PENSAR. ORGANIZAR. COMUNICAR. = CRECER JUNTOS



[Inscripción a la reunión - Zoom]

CLASE ANTERIOR

1 Del producto cruz a una transformación lineal



- Definimos $T(x) = a \times x$.
- Verificamos que preserva suma y producto por escalares.
- Toda transformación lineal puede representarse mediante una matriz.

Resultado

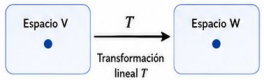
Construcción de la matriz antisimétrica $[a]_\times$.

$$[a]_\times = \begin{pmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{pmatrix}$$

Relación final

$$a \times x = [a]_\times x$$

2 Transformaciones lineales y aplicaciones



- Una transformación lleva vectores de un espacio a otro.
- La matriz representa completamente esa transformación.
- Distintas aplicaciones utilizan la misma idea matemática.

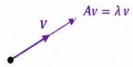
Ejemplos

- Visión por computadora
- Bases de datos
- Embeddings y NLP
- Transformaciones geométricas

3 Autovectores y autovalores

Algunas direcciones permanecen invariantes.

Sólo cambia su longitud.



Ecuación fundamental

$$Av = \lambda v$$

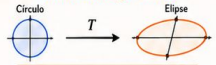
- v: dirección especial (autovector)
- λ: factor de escala (autovalor)

Interpretación

$\lambda > 1$	Se expande en la dirección v
$0 < \lambda < 1$	Se comprime en la dirección v
$\lambda = 1$	No cambia su longitud

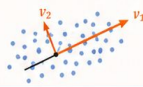
4 Geometría y análisis de datos

Transformaciones geométricas



Los ejes principales son autovectores.

PCA (Análisis de Componentes Principales)



- La matriz de covarianza Σ posee autovectores.
- Los autovalores λ_i representan la varianza.

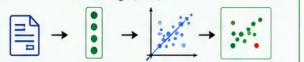
Idea clave

La matemática es la misma; cambia la matriz estudiada.

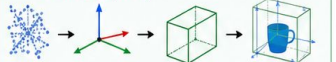
Transformación lineal	matriz A
PCA	matriz de covarianza Σ

5 Aplicaciones modernas de IA

Embeddings (CLIP)



Visión 3D (Objectron)



Idea final

Los autovectores aparecen en Álgebra Lineal, Estadística y Aprendizaje Automático.

Representar transformaciones mediante matrices → encontrar direcciones invariantes → analizar datos → construir aplicaciones modernas de IA.

CLASE DE HOY

Introducción a NumPy

1 ¿POR QUÉ EXISTE NUMPY?



Las listas de Python son flexibles, pero no están optimizadas para cálculos numéricos intensivos.

2 LISTAS vs ARRAYS

[1, "a", 3.5] (lista)

1.0	2.0	3.0

(ndarray)

Memoria dispersa, datos heterogéneos.

Memoria contigua, datos homogéneos. Más eficiente.

3 OPERACIONES CON NUMPY

1	2	3

* 2

↓

2	4	6

Las operaciones se aplican elemento por elemento de forma vectorizada: más rápido y sin bucles.

4 TIPOS DE DATOS (dtype)

- int32
- float64
- bool
- complex64
- ...

- Definen el tipo de dato.
- Afectan la precisión.
- Impactan en la memoria y el rendimiento.

5 FUNCIONES CLAVE DE NUMPY

Crear arrays | Estadística (mean, std, ...) | Álgebra lineal

Funciones matemáticas | Otras utilidades (random, reshape, ...)

6 MULTIDIMENSIONALIDAD DE NUMPY

0D	1D	2D	3D+
Escalar	Vector	Matriz	Tensor

Un ndarray puede representar desde un escalar hasta un tensor. Base para imágenes, video e IA.

IDEA CLAVE: NumPy permite trabajar con datos numéricos de forma eficiente, rápida y escalable, desde lo más simple hasta lo más complejo.

¿Por qué existe NumPy?

Del mundo de las listas al cálculo científico

Python tiene LISTAS

```
lista = [1, 2, 3, 4]
```

- Muy flexibles
- Pueden guardar cualquier tipo de dato
- No están optimizadas para cálculos numéricos intensivos



Cuando trabajamos con grandes cantidades de datos y operaciones matemáticas complejas...



Necesitamos herramientas más rápidas y eficientes para el cálculo científico.

Aquí es donde entra NUMPY

```
array = np.array([1, 2, 3, 4])
```

- Maneja vectores y matrices n-dimensionales (ndarray)
- Operaciones numéricas optimizadas en C
- Base de muchas librerías de Ciencia de Datos e IA

1.0	2.0	3.0	4.0

Datos numéricos homogéneos, almacenados de forma contigua en memoria → Más velocidad

NumPy es la base del ecosistema científico de Python

- **NumPy** — Cálculo numérico
- **Pandas** — Manipulación de datos
- **SciPy** — Cálculo científico avanzado
- **Scikit-Learn** — Machine Learning
- **TensorFlow** — Deep Learning
- Y muchas más

IDEA CLAVE: NumPy convierte Python en una herramienta de cálculo científico de alto rendimiento.

LISTAS DE PYTHON vs ARRAYS DE NUMPY

Principales diferencias

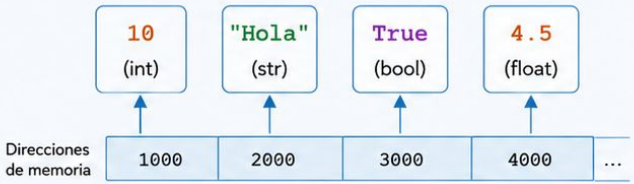
LISTAS (list)

Colección general de objetos

```
lista = [10, "Hola", True, 4.5]
```

Puede contener diferentes tipos de datos

Almacenamiento en memoria



Cada elemento es un objeto independiente y puede estar en cualquier parte de la memoria

Características clave

- Muy flexible
- Puede mezclar tipos de datos
- Operaciones elemento a elemento requieren bucles

	LISTAS (list)	ARRAYS DE NUMPY (ndarray)
Tipo de colección	Colección general de objetos	Arreglo numérico homogéneo
Tipos de datos	Puede contener diferentes tipos de datos	Todos los elementos son del mismo tipo (float)
Rendimiento	Cada elemento es un objeto independiente y puede estar en cualquier parte de la memoria	Datos almacenados de forma contigua en memoria → mayor velocidad y eficiencia
Almacenamiento en memoria	Operaciones elemento a elemento requieren bucles	Operaciones vectorizadas automáticas y muy rápidas
Operaciones	Muy flexible; Puede mezclar tipos de datos	Optimizado para cálculos numéricos; Mismo tipo de dato para todos

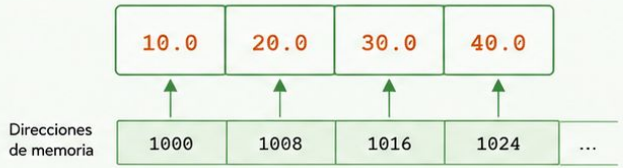
ARRAYS DE NUMPY (ndarray)

Arreglo numérico homogéneo

```
array = np.array([10.0, 20.0, 30.0, 40.0])
```

Todos los elementos son del mismo tipo (float)

Almacenamiento en memoria



Datos almacenados de forma contigua en memoria → mayor velocidad y eficiencia

Características clave

- Optimizado para cálculos numéricos
- Mismo tipo de dato para todos
- Operaciones vectorizadas automáticas y muy rápidas

IDEA CLAVE: Las listas son excelentes para flexibilidad y datos heterogéneos. | NumPy Arrays son ideales para cálculos numéricos intensivos y análisis de datos.

OPERACIONES: PYTHON vs NUMPY

NumPy permite realizar operaciones matemáticas sobre todos los elementos del array de forma eficiente.

LISTAS DE PYTHON

VS

ARRAYS DE NUMPY

Multiplicación

```
>>> [1, 2, 3] * 2
```

Resultado:

```
[1, 2, 3, 1, 2, 3]
```

En las listas, el operador * repite la lista, NO multiplica los números.

```
 [1, 2, 3]
 * 2
 ---------
= [1, 2, 3,
   1, 2, 3]
```

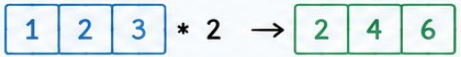
Multiplicación

```
>>> np.array([1, 2, 3]) * 2
```

Resultado:

```
[2, 4, 6]
```

En NumPy, el operador * realiza la multiplicación elemento por elemento.



Otras operaciones con listas

```
>>> [1, 2, 3] + [4, 5, 6]
Resultado: [1, 2, 3, 4, 5, 6]
```

Las operaciones son limitadas y no son matemáticas elemento por elemento.

Otras operaciones con arrays

```
>>> np.array([1, 2, 3]) + np.array([4, 5, 6])
Resultado: [5, 7, 9]
```

Operaciones matemáticas aplicadas elemento por elemento:

`+` `-` `*` `/` `**`

VECTORIZACIÓN



MUCHO MÁS RÁPIDO

NumPy realiza las operaciones internamente en código optimizado (C), sin necesidad de bucles explícitos de Python.

```
for i in range(n):
    c[i] = a[i] * b[i]
```

```
c = a * b
```

IDEA CLAVE: NumPy transforma a Python en una herramienta poderosa para cálculos numéricos y científicos.

La multidimensionalidad de NumPy

Un ndarray puede representar datos de cualquier dimensión.

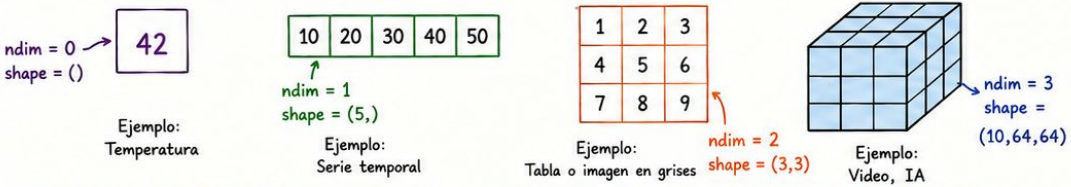
La dimensión (ndim) indica cuántos ejes (axes) tiene el arreglo.

Idea clave

En NumPy, todos estos objetos son ndarray; lo único que cambia es el número de dimensiones (ndim) y su forma (shape). Esta uniformidad permite escribir el mismo tipo de operaciones para datos simples o altamente multidimensionales.

De menor a mayor dimensión

Escalar (0D) → Vector (1D) → Matriz (2D) → Tensor (3D+)



Ejemplos en NumPy

Escalar (0D)

```
import numpy as np
escalar = np.array(42)
print(escalar)
print(escalar.ndim)     # 0
print(escalar.shape)    # ()
```

```
42
0
()
```

Vector (1D)

```
vector = np.array([10,20,30,40])
print(vector)
print(vector.ndim)      # 1
print(vector.shape)     # (4,)
```

```
[10 20 30 40]
1
(4,)
```

Matriz (2D)

```
matriz = np.array([
    [1,2,3],
    [4,5,6]
])
print(matriz.ndim)      # 2
print(matriz.shape)     # (2,3)
```

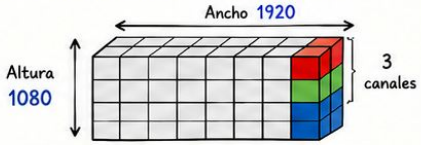
```
2
(2, 3)
```

Tensor (3D)

```
tensor = np.zeros((5,4,3))
print(tensor.ndim)      # 3
print(tensor.shape)     # (5,4,3)
```

```
3
(5, 4, 3)
```

Ejemplo práctico — Imagen RGB



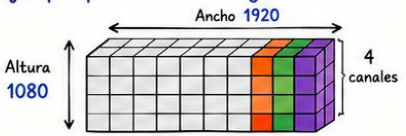

```
imagen_rgb = np.zeros((1080,1920,3),
                      dtype=np.uint8)
print(imagen_rgb.shape)
# (1080, 1920, 3)
```

Cada píxel almacena:

[255, 120, 60]

R G B

Ejemplo práctico — Imagen RGB-D




```
imagen_rgbd = np.zeros((1080,1920,4),
                       dtype=np.uint16)
print(imagen_rgbd.shape)
# (1080, 1920, 4)
```

Cada píxel almacena:

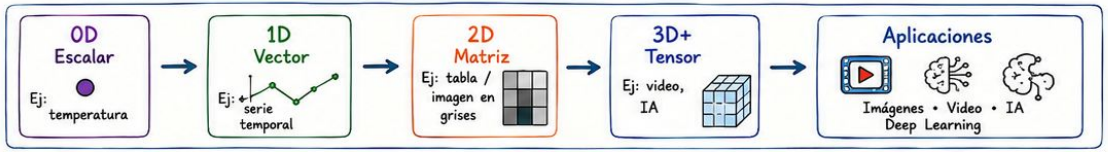
[R, G, B, Depth]

Se utiliza en:

- Robótica
- Vehículos autónomos
- Intel RealSense
- Kinect
- Visión por computadora

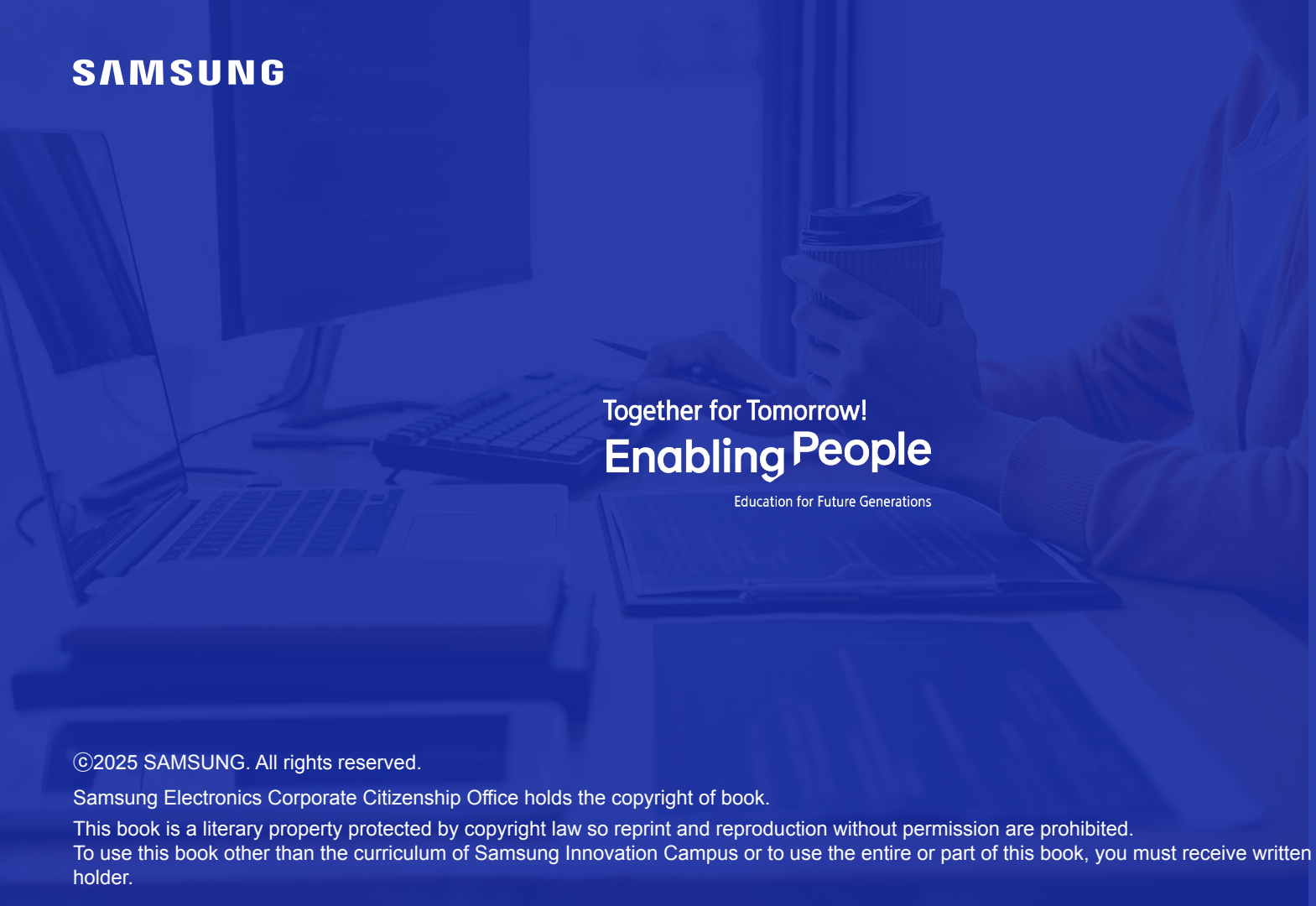
Idea clave

Un ndarray es flexible y potente: mismo tipo de objeto, cualquier dimensión.



En NumPy:

- ✓ Todos son ndarray
- ✓ Cualquier dimensión
- ✓ Operaciones vectorizadas
- ✓ Eficiente y potente



ⓒ2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.